

Manual Técnico

Analizador Léxico del lenguaje **PromptZal**

Nombre: Emanuel Sosof **Carnet:** 202330664

Fecha: 17 / 08 / 2026

1. Introducción

PromptZal es un lenguaje de alto nivel, basado en **prompts** (instrucciones en lenguaje natural), diseñado para interactuar con modelos de inteligencia artificial. Un programa PromptZal declara un agente con un contexto y ejecuta comandos de IA como **PREGUNTAR**, **GENERAR**, **RESUMIR**, entre otros.

Este proyecto implementa un **analizador léxico** (tokenizador) escrito **manualmente en Java**, sin usar generadores automáticos (JFlex, JLex, Lex) ni expresiones regulares de Java. Su función es leer un archivo **.pz**, dividir el texto en **tokens** (piezas mínimas con significado), clasificarlos, detectar **errores léxicos** y generar **reportes** en consola y en HTML.

2. Estructura del proyecto

Proyecto **Maven** en Java 21. Cada clase tiene una responsabilidad única:

Archivo	Responsabilidad
Token.java	Modelo de un token: guarda lexema (texto), tipo , fila y columna , con sus métodos de acceso (getters).
Diccionario.java	Conjuntos oficiales de palabras del lenguaje (directivas, reservadas, comandos, funciones, conectores) y el método <code>clasificar()</code> que dice si una palabra es de un tipo o es identificador.
Tokenizador.java	Motor del análisis : recorre el texto carácter por carácter, forma tokens y detecta errores.
Reportes.java	Imprime la tabla de tokens y errores en consola, y construye el reporte HTML .
PromptZal.java	Punto de entrada : menú principal que pide la ruta del archivo, le llama al tokenizador y muestra/guarda los reportes.
pom.xml	Configuración Maven: versión de Java (21) y plugins para compilar y empaquetar el <code>.jar</code> ejecutable.
casos/validos/ y casos/errores/	5 casos de prueba: 3 programas válidos y 2 con errores léxicos.

3. Lista de tokens del lenguaje

El análisis léxico reconoce las siguientes categorías de tokens:

3.1 Directivas

Directiva

Token	Descripción	Ejemplo
@modelo	Indica el modelo de IA que usará el agente.	@modelo "claude-sonnet-4-6"
@rol	Define el rol o perfil del agente.	@rol "analista financiero"
@formato	Especifica el formato de salida esperado.	@formato "json"

3.2 Palabras reservadas

Palabra reservada

Token	Descripción	Ejemplo
AGENTE	Declara el nombre de un agente.	AGENTE analista
contexto	Asigna el texto de contexto del agente.	contexto = "Eres un analista"
variable	Declara una variable que guarda un valor o resultado.	variable ventas = CARGAR("ventas.csv")
EJECUTAR	Ejecuta una acción del agente.	EJECUTAR analista
EXPORTAR	Exporta un resultado a un destino.	EXPORTAR resultado -> archivo

3.3 Comandos de IA

Comando de IA

y función

Función

Token	Descripción	Ejemplo
PREGUNTAR	Hace una pregunta al modelo.	PREGUNTAR "Que paso?"
GENERAR	Genera contenido nuevo.	GENERAR "poema"
RESUMIR	Resume un documento.	RESUMIR informe COMO "resumen.pdf"
ANALIZAR	Analiza datos o texto.	ANALIZAR datos
TRADUCIR	Traduce contenido.	TRADUCIR texto EN "español"
CLASIFICAR	Clasifica elementos.	CLASIFICAR comentarios EN "positivo"
EXTRAER	Extrae información.	EXTRAER datos
CARGAR	Función que lee un archivo externo.	CARGAR("ventas.csv")

3.4 Conectores

Conector

Token	Descripción	Ejemplo
SOBRE	Enlaza un comando con el dato sobre el que actúa.	PREGUNTAR "... " SOBRE ventas
DESDE	Indica origen.	ANALIZAR datos DESDE archivo
EN	Indica destino, idioma o lista.	CLASIFICAR comentarios EN "positivo"
COMO	Indica la forma de salida.	RESUMIR informe COMO "resumen.pdf"
->	Flecha: asigna o envía un resultado a un destino.	EXPORTAR resultado -> archivo

3.5 Operadores

Operador

Token	Descripción	Ejemplo
=	Asignación.	contexto = "Eres un analista"
+	Concatenación.	variable total = 1 + 2

3.6 Delimitadores

Delimitador

Token	Descripción	Ejemplo
{ }	Delimitan el bloque de declaración del agente.	AGENTE analista { ... }
()	Delimitan los argumentos de una función.	CARGAR("ventas.csv")
,	Separa elementos de una lista.	CLASIFICAR x EN "a", "b"
" "	Comillas: delimitan los literales de cadena.	"Eres un analista"

3.7 Identificadores

Identificador

Regla	Descripción	Ejemplos
Letra o <code>_</code> seguido de letras, dígitos o <code>_</code>	Nombres libres que el usuario elige (nombres de agentes, variables, destinos). Cualquier palabra que no esté en el diccionario se clasifica como identificador.	<code>analista</code> , <code>ventas</code> , <code>reporte</code> , <code>mi_variable</code>

3.8 Literales

Literal

Tipo	Descripción	Ejemplos
Cadena	Texto entre comillas dobles.	<code>"Eres un analista"</code> , <code>"ventas.csv"</code>
Entero	Solo dígitos.	<code>100</code> , <code>2</code>
Decimal	Dígitos con punto y dígitos después.	<code>2.5</code> , <code>99.99</code>

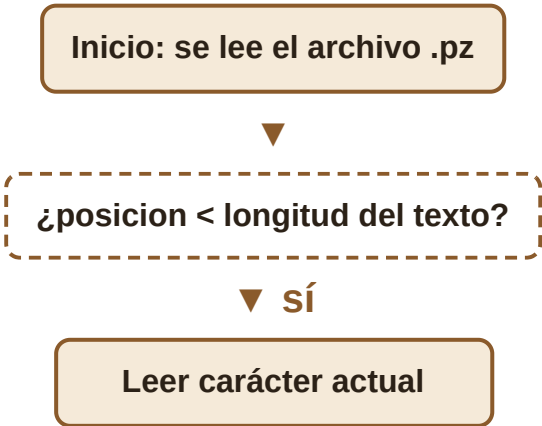
3.9 Comentarios

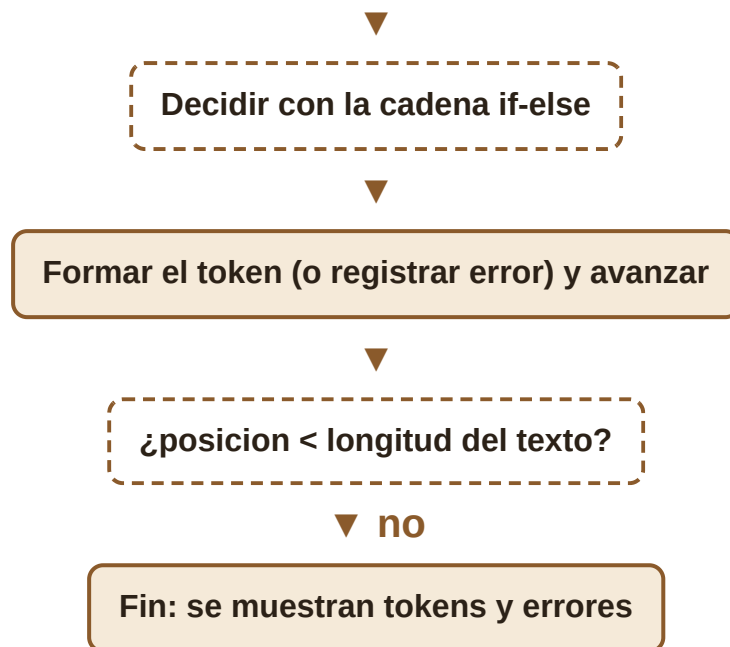
Se ignoran

Sintaxis	Descripción	Ejemplo
<code>// texto</code>	Comentario de línea: se ignora hasta el fin de línea.	<code>// esto es un comentario</code>
<code>/* texto */</code>	Comentario de bloque: se ignora todo lo que abarca.	<code>/* contexto\n del agente */</code>

4. Recorrido del análisis léxico

El analizador trabaja **carácter por carácter** sobre el contenido del archivo. El método `analizar(String contenido)` ejecuta un bucle `while` que avanza por el texto con una variable `posicion`, y en cada vuelta decide, con una cadena de `if-else`, qué tipo de token va a formar.





4.1 Tabla de decisiones del bucle principal

Carácter actual	Acción	Resultado
Espacio, tab, salto de línea	<code>avanzar()</code>	Se ignora (no genera token)
Letra o <code>_</code>	<code>leerPalabra()</code>	Reservada / comando / función / conector / identificador
Dígito	<code>leerNumero()</code>	Entero o decimal
<code>@</code>	<code>leerDirectiva()</code>	Directiva
<code>"</code>	<code>leerCadena()</code>	Literal cadena (o error si no se cierra)
<code>=</code>	<code>tokenUnCaracter("Operador asignacion")</code>	Operador asignacion
<code>+</code>	<code>tokenUnCaracter("Operador concatenacion")</code>	Operador concatenacion
<code>{ } () ,</code>	<code>tokenUnCaracter("Delimitador")</code>	Delimitador
<code>-</code> seguido de <code>></code>	Token directo	Conector <code>-></code>
<code>/</code>	<code>comentario()</code>	Se ignora <code>//</code> o <code>/* */</code> (o error)
Cualquier otro	Registrar error	Error léxico: caracter no reconocido

4.2 Métodos del Tokenizador

Método	Qué hace
<code>avanzar()</code>	Mueve <code>posicion</code> al siguiente carácter y actualiza <code>fila</code> / <code>columna</code> : si el carácter es salto de línea, suma una fila y reinicia columna a 1; si no, solo suma columna.
<code>leerPalabra()</code>	Acumula letras, dígitos y <code>_</code> hasta un separador, luego pregunta al diccionario: si existe, es reservada/comando/etc.; si no, es identificador .
<code>leerNumero()</code>	Acumula dígitos; si encuentra un <code>.</code> seguido de dígitos, forma un decimal; si no, un entero.
<code>leerDirectiva()</code>	Toma el <code>@</code> y luego las letras; clasifica con el diccionario.
<code>leerCadena()</code>	Acumula hasta la comilla de cierre; si encuentra un salto de línea antes, la cadena quedó sin cerrar y se registra el error.
<code>comentario()</code>	Si el siguiente es <code>/</code> ignora la línea; si es <code>*</code> ignora hasta <code>*/</code> (o registra error de bloque sin cerrar); si es otro, es un <code>/</code> suelto y da error.
<code>tokenUnCaracter(tipo)</code>	Crea el token de un solo carácter con el tipo dado y avanza.

Fila y columna: cada token guarda la posición donde **empieza**. Por eso los métodos capturan `int f = fila, c = columna;` **antes** de leer, ya que `avanzar()` va moviendo esas variables mientras se forma el token.

5. Manejo de errores léxicos

El analizador detecta 3 tipos de errores léxicos y **no se detiene**: reporta el error y continúa analizando hasta el final del archivo.

Error	Mensaje	Ejemplo que lo provoca
Carácter no reconocido	Error lexico: caracter no reconocido	<code>5 & 3</code> , <code>10 - 2</code> , <code>ventas.csv</code>
Cadena sin cerrar	Error lexico: cadena sin cerrar	<code>PREGUNTAR "hola</code> (sin comilla de cierre)
Comentario de bloque sin cerrar	Error lexico: comentario de bloque sin cerrar	<code>/* esto nunca se cierra</code>

El análisis continúa: al encontrar un error, el tokenizador lo guarda en una lista aparte y **avanza** con `avanzar()`. La regla de oro del bucle es que **toda rama avanza al menos un carácter**, por lo que jamás se queda en un bucle infinito.

Importante: los errores que NO son de esta práctica son los **sintácticos** (piezas válidas mal ordenadas): llaves de más o de menos, `variable` sin nombre, etc. El analizador léxico solo clasifica piezas; no verifica la estructura del programa.

6. Reportes generados

6.1 Consola

Tras analizar, se imprime la tabla de tokens (`# | Lexema | Tipo | Fila | Columna`) y, al final, la sección de errores léxicos (o «ninguno»).

6.2 HTML

Se genera automáticamente un reporte **HTML** con el mismo nombre que el archivo analizado y extensión `.html`, **en la misma carpeta** donde está el `.pz`. Contiene dos pestañas: **Tokens** y **Errores**, tarjetas de resumen y tablas detalladas. Se muestra en consola la ruta exacta donde quedó guardado.

```
Reporte HTML generado en: /home/mrk-ema/NetBeansProjects/PromptZal/casos/validos/agente_an
```